

Enhancing Retrieval-Augmented Generation Quality via Test-Time Scaling with Beam Search Decoding

Cong Nga Ha^{1*}, Ngoc Hai Dang Dao¹, Van Ket Luong¹,
Minh Diem Vu¹

¹ Group 9 - Modern Informatic Problem, VNU University of Engineering and Technology (VNU-UET), Hanoi, Vietnam .

*Corresponding author(s). E-mail(s): 21020127@vnu.edu.vn;
Contributing authors: 21020301@vnu.edu.vn; 22021182@vnu.edu.vn;
21020012@vnu.edu.vn;

Abstract

Retrieval-Augmented Generation (RAG), such as **LightRAG**[1], **GraphRAG**[2], **Self-RAG**[3], etc, have proved their potential in improving the factual accuracy of Large Language Models (LLMs) by integrating external knowledge. The OPEN-RAG framework enhances retrieval-augmented reasoning by combining adaptive multi-hop retrieval with reflection-driven generation. From the aforementioned techniques, OPEN-RAG encourages the model to refine its own reasoning methods. This mechanism guides the LLM to verify the relevance of retrieved evidence accurately and promotes more deliberate multi-hop inference across multiple pieces of information. As a result, OPEN-RAG improves the reliability and factual consistency of RAG, namely in complex reasoning scenarios where supporting documents must be coherent. However, a persistent challenge occurred when provided with highly relevant or fully correct retrieved contexts; RAG systems often produce suboptimal or incorrect results because of limitations in the generation or decoding phase [4] [5]. To improve the robustness of this decoding stage, this study proposes a simple yet effective enhancement to the OPEN-RAG framework through the use of **Beam Search Decoding**. Inspired by test-time scaling [6] [7], we hypothesize that allowing the model to explore a wider space of candidate sequences during decoding can refine the output quality and mitigate the limitations of conventional greedy decoding. Our experiments show consistent improvements across key RAG metrics, with a beam width of $k = 6$ performing the best among the configurations we evaluated: the F1 score increased from **0.7871** to **0.7901**, and Exact Match (EM) improved from **0.6324** to **0.6371**. These results confirm that a simple increase in decoding effort, achieved through beam-search decoding, acts as an effective form of test-time scaling. A small beam size (typically $k = 2$ to $k = 6$) made the model generate locally fluent and globally optimal responses, meaning they are more accurate and supported by the retrieved text. This advanced search technique introduces a small latency increase compared to greedy search, proving response quality improvement is achieved without a huge degradation in responsiveness. The code implementation and experimental configurations are available at <https://github.com/HaCongNga/G9-OpenRAG-EnhancedGeneration/tree/main>

Keywords: Retrieval-Augmented Generation, Large Language Models, Beam Search, Decoding Strategy, OPEN-RAG, Test-Time Scaling

1 Introduction

Large Language Models (LLMs) have opened up a range of applications and opportunities in the natural language processing domain with impressive capabilities in text generation, role playing, translation and problem solving etc[8][9]. However, it has one prominent intrinsic defect: hallucination which stems from its next token prediction design and its reliance on its parameters [4][10]. Since then, The **Retrieval-Augmented Generation (RAG)** paradigm has become the dominant method for mitigating this issue by providing the LLMs with external relevant context.

The **OPEN-RAG** framework is the state of the art method in the RAG paradigm [11][12][13] with the aim to transforming the "passive" context digestion of the LLMs to "active" use of the retrieved context by introducing reflection tokens and adaptive retrieval mechanism.

1.1 Problem Statement and Motivation

Despite OPEN-RAG's success, the problem of hallucinations still persists. One of the disadvantages of the framework is that it still utilizes the retrieved context poorly. Specifically, it ignores facts from the retrieval context leading to inaccurate responses even when the retrieval context is concise. This defect hints the problem in the decoding phase where the greedy decoding is not optimized for factual accuracy in the situations where multi-hop reasoning is required. Our goal is to improve the decoding process of the OPEN-RAG framework and build upon it to leverage its strengths. To achieve this goal, we explored a new test-time scaling method that can be integrated well with the RAG pipeline and helped enhance the reasoning capability of the system, especially in the multi-hop reasoning settings.

1.2 Proposed Solution: Beam Search Decoding

We propose the use of Beam-Search decoding in the generation phase of the LLMs. Beam-Search maintains k most promising paths in the decoding phase (with k is the beam size), so the resources in the retrieved context are better utilized and explored. It acts as a decoding-level test-time scaling.

1.3 Structure of the Paper

The paper is structured as follows: Section 2 introduces related work in the RAG and LLM decoding; Section 3 briefs about the OPEN-RAG framework; Section 4 proposes the integration and theoretical basis of Beam-Search. Section 5 details about the experimental setup and result analyses; Section 6 is about the conclusion of this study.

2 Related Works

2.1 Retrieval Augmented Generation

Retrieval Augmented Generation or RAG is a prominent technique to reduce the hallucinations phenomenon of the LLMs [14] [15] as it incorporates external relevant and up-to-date evidence and was used to feed to the LLMs along with the user queries. Given the retrieved contexts, the LLMs did not have to solely rely on its parametric knowledge and hence leading to more precise and correct responses [14] [15]. However, this strategy often breaks out when the retrieved contexts contain noise or the final answer needs to be synthesized from multiple documents.

Self-RAG [3] is one of the early works that adopt a multi-step strategy in the reasoning pipeline for RAG applications. It actively evaluates whether the retrieved contexts are adequate or not and may trigger additional retrieval rounds. It also introduces the idea of self-revision by judging its own generation. Building on these ideas, **OPEN-RAG** extends

the reflection tokens to a comprehensive vocabulary covering Retrieval, Relevance, Grounding, and Utility open-source models and introduces two key innovations. Second, it proposes a **PEFT-MoE (Parameter-Efficient Mixture-of-Experts)** architecture, which enables efficient further training of the base models to make them more robust to multi-hop reasoning settings.

2.2 Decoding Strategies and Reasoning

Decoding is the final phase in the generation process of the LLMs and it plays a pivotal role in deciding the quality and reliability of the responses. There are several variations in this phase such as **greedy decoding**, **top-k sampling**, and **nucleus sampling** and each of them offers advantages and disadvantages when it comes to determinism, speed, and diversity. However, these strategies are not explicitly optimized for multi-hop reasoning tasks. Recent research shows that by designing a custom strategy in the decoding phase, the LLMs can be guided to the desired path of the strategy. For example, Deng et al. [16] proposes the use of re-ranking over multiple reasoning paths to select the most evidence-grounded sequence path. Decoding has become a niche area for targeted generation.

2.3 Test-Time Scaling for Reasoning

Test-time scaling has been gaining traction recently with the simple idea: trading performance off additional computation during the inference phase.

Chain-of-Thought (CoT) [17] prompting is among one of the most powerful approach. Through producing intermediate reasoning steps in the thinking phase, the models gradually discover the correct reasoning paths and learn to decompose the tasks into multiple steps to tackle.

More recently, simple Test-Time Scaling (S1) [6] demonstrates that through controlling the tokens allocated in the generation phase by learning to stop the LLMs early or force the models to spend more time on hard challenges, we can boost the reasoning capabilities measurably.

On the other hand, the Roster of Experts (RoE) framework [18] introduces token-level rather than sequence-level scaling. It introduces randomness and stochasticity into the experts routing and successfully produces natural and diverse generation paths, through which the system can do an aggregation to select the best paths.

These developments showed that the LLMs can definitely benefit from the skillful scaling techniques.

3 Background: The OPEN-RAG Framework

This part gives details about the baseline and its key innovations.

3.1 PEFT Sparse MoE Architecture

Given the diverse domains and scenarios in the RAG system, especially in cases of multi-hop reasoning, Open-RAG [11] proposed to convert the dense model to mixture of experts architecture. The idea is that throughout the training process, each expert will specialize in one domain or functionalities and learning to combine their strength towards tackling the problem. Open-RAG "upcycled" the Feed-Forward Network (FFN) layers by attaching lightweight expert modules and a router. To further saving training costs, the framework proposed to use LoRA - a parameter-efficient finetuning method to finetune the LLMs and employed the sparse routing mechanism. These strategies allow the LLMs to:

- allocate more compute to "hard" tokens that deciding the reasoning paths.
- remain efficient to simpler queries with the selective experts chosen.

- achieve the performance of much bigger models without exploding the finetuning and inferencing costs.

3.2 Reflection Tokens

OPEN-RAG’s second innovation lies in the usage of **reflection tokens** to enhance the self-verification of the LLMs and hence reduce hallucinations in the multi-hop reasoning settings. Instead of directly answering the user questions, the model is forced to produce tokens to self-critique its own response (relevance, grounding, utility) and give explicit judgements about whether retrieval is needed.

The reflection tokens added to the vocabularies include:

- **[RT]** / **[NoRT]**: Demonstrate whether retrieval is necessary or not. If **[RT]**, the retriever will retrieve external contexts else the LLMs generates the answer by itself.
- **[Fully Supported]** / **[Partially Supported]** / **[No Support]**: Self-evaluate how the response y_t is grounded in the evidence, providing an insight into the contexts and the awareness of the LLMs
- **[Relevant]** / **[Irrelevant]**: Evaluates the quality of the retrieved contexts s_t .
- **[Relevant]** / **[Irrelevant]**: Evaluates the quality of the retrieved contexts s_t .
- **[U:1]**–**[U:5]**: A utility score that reflects how useful the model finds its own responses.

Each of the aforementioned tokens added new lens on the reasoning process of the LLMs. The LLMs now is not just a token by token prediction model but a self-critic model that is able to reason about the retrieved contexts. This introspection has proved to improve the reliability of the RAG systems.

4 The Proposed Method: Beam Search Enhancement

4.1 Beam Search Formalization

Our goal is to find the sequence $Y = (y_1, y_2, \dots, y_L)$ that maximizes the conditional probability $P(Y|X)$, where X is the concatenated input.

$$Y^* = \arg \max_Y P(Y|X)$$

Greedy decoding approximates this by maximizing $P(y_t|y_{1:t-1}, X)$ at each step t . Beam Search provides a better heuristic approximation by maintaining a set of k sequences that maximize the log-probability:

$$\text{Score}(Y) = \frac{1}{|Y|^\alpha} \sum_{t=1}^{|Y|} \log P(y_t|y_{1:t-1}, X) \quad (1)$$

where α is the length normalization hyperparameter.

4.2 Integration Details

The Beam Search decoding was used to generate the response y_t for each context s_t . It enables LLMs to explore broader reasoning paths, leading to more accurate answers.

The critical step is line 5, where the enhanced decoding explores a broader space. A successful token path in a RAG setting is one that strictly adheres to the context. By exploring k paths, we increase the probability of finding a sequence that maximizes $P(Y|X)$ subject to the factual constraints imposed by s_i .

As can be seen in Algorithm 2 maintains top-k beam sequences at each decoding step in line 6.

Algorithm 1 OPEN-RAG Generation with Beam Search (k)

Require: Query q , Contexts $S = \{s_1, \dots, s_m\}$, Model M , Beam Width k

- 1: $C \leftarrow \emptyset$ ▷ Set of candidate answers
- 2: **for** each context $s_i \in S$ **do**
- 3: $P_i \leftarrow$ Run M to predict Reflection Tokens for s_i
- 4: $\text{Prompt}_i \leftarrow q \oplus s_i \oplus P_i$
- 5: $y_i \leftarrow \text{BeamSearch}(\text{Prompt}_i, M, k)$ ▷ Enhanced generation
- 6: $U_i \leftarrow$ Run M to predict Utility $[U : j]$ for y_i
- 7: $C \leftarrow C \cup \{(y_i, U_i, P_i)\}$
- 8: **end for**
- 9: $y_{\text{final}} \leftarrow \text{RankAndSelect}(C)$ ▷ Using OPEN-RAG’s weighted ranking
- 10: **return** y_{final}

Algorithm 2 BeamSearch(Prompt, M, k)

- 1: $\text{beams} \leftarrow \{(\text{Prompt}, 0)\}$
- 2: **for** $t = 1$ **to** max_len **do**
- 3: $\text{cands} \leftarrow \emptyset$
- 4: **for** each $(seq, score)$ in beams **do**
- 5: $p \leftarrow M.\text{NextTokenProbs}(seq)$
- 6: **for** each $(token, prob)$ in $\text{TopK}(p, k)$ **do**
- 7: $\text{cands} \leftarrow \text{cands} \cup \{(seq + token, score + \log prob)\}$
- 8: **end for**
- 9: **end for**
- 10: $\text{beams} \leftarrow \text{TopK}(\text{cands}, k)$
- 11: **end for**
- 12: **return** $\text{best}(\text{beams})$

4.3 Test-Time Scaling and Efficiency

The increased search depth and breadth directly translate into improved quality but also increased latency. Theoretically, employing a beam size or sampling factor of k introduces a relative computational cost increase of $\mathcal{O}(k)$ compared to simple greedy search. However, highly efficient batched decoding makes k parallel beams execute simultaneously on GPU hardware. By leveraging the parallel nature of the matrix multiplications and shared Key-Value (KV) cache mechanisms, the latency overhead for increasing k becomes negligible until k is very large. This means the additional computation for k beams can be absorbed at the cost of a marginally larger memory footprint, effectively turning the test-time search into a near-constant-factor latency operation. We therefore investigate $k = 4$ and $k = 6$ to determine the optimal balance between accuracy and this practical, rather than theoretical, computational cost.

5 Experiments

The experiments are designed to test the following core research questions:

- *RQ1 (Core)*: Does increasing the decoding search space via Beam Search significantly improve factual accuracy (F1, EM) in the OPEN-RAG framework?
- *RQ2 (Cost)*: What is the cost incurred when increasing the beam width from $k = 1$ to $k = 6$?
- *RQ3 (Behaviour analysis)*: What is the behavior of the RAG system enhanced by beam-search?

5.1 Tasks and Datasets

We briefly describe the tasks and datasets for the evaluations of the RAG system in the literature and then give details about the dataset used in our study. Specifically, there are three main tasks commonly used by their complexity:

- **Single-hop Short-form Tasks:** These require extracting a single, short answer from the retrieved documents. Several popular datasets under this category are PopQA, TriviaQA, Pubhealth...
- **Single-hop Long-form Generation Tasks:** These evaluate the models on generating comprehensive and descriptive answers or long-form question answering.
- **Multi-hop Reasoning Tasks:** These tasks are considered to be the most challenging, requiring models to realize complex relationships or synthesize across retrieved documents. Key examples include **HotpotQA** [13], MuSique Ans, and 2WikiMultihopQA.

We focus our experiments on the highly demanding **multi-hop reasoning** task using the **HotpotQA** dataset [13] as in OpenRAG [11]. HotpotQA composes of **7405 question-answer pairs** sourcing from Wikipedia and ranging across topics of politics, natural sciences, mathematics, common knowledge etc and testing the ability of the models to discover hidden facts and knowledge given from the retrieved documents.

The complex nature of HotpotQA provides a demanding environment to evaluate how effectively our proposed Beam Search decoding strategy improves factual alignment and mitigates the generation of ungrounded or incomplete responses compared to the standard greedy decoding.

5.2 Configuration and Evaluation

5.2.1 Metrics

We utilize standard RAG evaluation metrics: F1 Score, Exact Match (EM), Precision, and Recall [19].

Exact Match (EM). To compute the Exact Match, we compare the predicted answer and the ground truth token by token. The metric is 0 if a single token from the predicted answer is different from the ground truth (after normalization or removing the stopwords ...), else the metric is 1. EM is widely used in tasks that have require the LLMs to generate short and concise answers such as people’s names or places etc. The drawback of it is its lack of flexibility in measuring the semantic correctness of the ground truth and the predicted answers.

Mathematically, given a prediction p and ground-truth set G :

$$\text{EM}(p, G) = \begin{cases} 1, & \text{if } p = g \text{ for any } g \in G, \\ 0, & \text{otherwise.} \end{cases}$$

Precision. Precision measures how many tokens out of the predicted answer actually appear in the ground truth answers. Precision, therefore, is a metric that represents the conciseness and relevance of the generated answers. Formally, the Precision is calculated as:

$$\text{Precision} = \frac{|p \cap g|}{|p|}.$$

Recall. Recall neasures how many tokens in the correct answers are successfully recovered by the model. This metric represents the completeness and truthfulness of the generated answers. Mathematically, the Recall formular is:

$$\text{Recall} = \frac{|p \cap g|}{|g|}.$$

F1 Score. F1 Score. F1 is a combination of the two metrics Precision and Recall and balances the measurement of being concise (not producing irrelevant tokens) and being complete (not missing important information). The formula is:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

5.2.2 Configurations

- **Baseline:** Greedy Decoding ($k = 1$).
- **Beam Search** ($k = 4$): Medium-level search effort.
- **Beam Search** ($k = 6$): High-level search effort.
- **Length Normalization** (α): We choose $\alpha = 1.0$ to penalize the models from choosing the short decoding paths. With $\alpha = 1.0$, the beam score is normalized by its sequence length.

5.2.3 Hardware and Computational Cost

To perform the experiments in this study, we rent 2 NVIDIA RTX 4090 (22 GB VRAM each). Each method takes one day (24 hours) to finish the evaluation on the HotpotQA dataset which comprises of 7405 challenging questions and tasks. The data type is chosen at bfloat16 to save the memory and computation cost without affecting the overall performance.

5.3 Quantitative Results

The experimental results are summarized in Figure 1.

5.4 Analysis and Discussion

5.4.1 Answer to RQ1 - Factual Accuracy Improvement

As demonstrated in Figure 1, the use of beam-search decoding yields measurable gains across metrics.

- **F1 and Exact Match (EM):** Both metrics increase when k is increased. The best scores are achieved at $k = 6$, with F1 peaking at **0.7901** and EM at **0.6371**. This improvement validates the concept of applying test-time scaling at the generation phase: forcing the model to explore wider paths successfully identifying sequences with better factual alignment.
- **Recall:** The most substantial improvement is observed in Recall, which rises significantly from the baseline of 0.7778 to **0.7857** at $k = 6$.
- **Precision:** Precision remains competitive, maintaining a score of 0.7945 at $k = 6$ compared to the baseline of 0.7967.

These metrics will further be analyzed in the following sections of Behaviour Analysis.

5.4.2 Visualizing Performance Gains

To better illustrate the changes across configurations, we plot the performance metrics in Figure 1.

The visualization clearly shows the upward trend for F1 and EM as k increases, confirming the value of the deeper search.

5.4.3 Answer to RQ2 - Cost Analysis and Efficiency

One of the natural questions that arises when it comes to applying test-time scaling for RAG-based applications is the trade-off between performance gains versus the inference cost and latency of the responses. Interestingly, beam-search decoding proves to be a highly

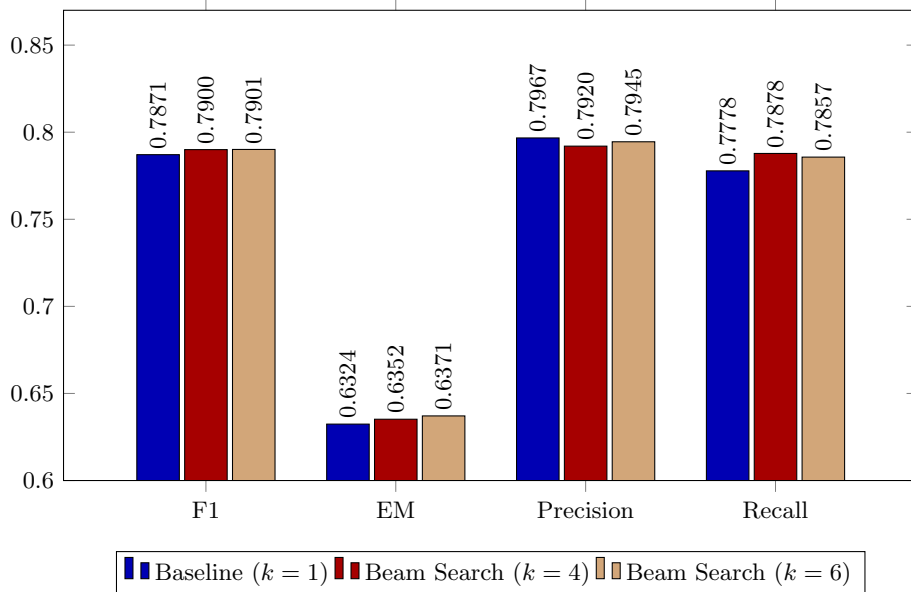


Fig. 1: Comparison of key RAG metrics across Baseline (Greedy), Beam Search $k = 4$, and Beam Search $k = 6$. Numbers are rotated 90° for perfect readability without any overlap.

efficient, low-cost, low-latency, post-training performance boost method. The efficiency of beam-search decoding is achieved through the implementation of batched decoding and the use of KV caching, where k beams of sequences are processed in parallel during the generation phase. These characteristics position the proposed method as valuable in high-stakes domains such as legal analysis, complex question answering for healthcare applications, or financial fraud analysis ... where factual accuracy is critical.

5.4.4 Answer to RQ3 - Behaviour Analysis

Given the significant rise in Recall, which measures the correlation between the generated response and the ground truth and the slight decrease in Precision, which calculates how many tokens in the generated answer actually appear in the ground truth answer, we can infer that the RAG system with beam-search decoding selecting tokens that align better with the correct answer albeit a side effect of making the response become more verbose.

6 Conclusion

This study demonstrated a simple and novel approach that beam-search decoding is an efficient and effective test-time scaling approach under the RAG system that requires reasoning and multi-hop retrieval. Through expanding the token sequence space, beam search encourages the LLMs to explore the most suitable tokens for answering complex queries. By leveraging batch inference, beam-search decoding was shown to help improve accuracy while introducing negligible latency with the recommended beam-size being 6 which yields the best results under our experiments with **F1 (0.7901)** and **Exact Match (0.6371)**.

7 Limitations and Future Work

Despite a robust result shown, the study was restricted by the computational budget and there are still many promising directions to improve the proposed method further. One idea is to explicitly force the LLMs to generate thoughts before answering the multi-hop reasoning questions with beam-search in the thinking phase. This will help the model to explore wider

reasoning paths before answering questions. Second, we can finetune the LLMs further to make it learn in the latent reasoning traces so that it can adapt better for the reasoning tasks.

References

- [1] Guo, Z., Xia, L., Yu, Y., Ao, T., Huang, C.: LightRAG: Simple and Fast Retrieval-Augmented Generation. arXiv preprint arXiv:2410.05779 (2024)
- [2] Han, H., Wang, Y., Shomer, H., Guo, K., Ding, J., Lei, Y., Halappanavar, M., Rossi, R.A., Mukherjee, S., Tang, X., He, Q., Hua, Z., Long, B., Zhao, T., Shah, N., Javari, A., Xia, Y., Tang, J.: Retrieval-augmented generation with graphs (graphrag). arXiv preprint arXiv:2501.00309 (2025)
- [3] Itani, A., Mallen, D., Yang, C., Zhang, J., Aydin, S., Xu, G., Lin, W., Zhu, Z., Lee, C.-W., Zhang, P., Wu, Z., Zhang, C., Sun, Y.: Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. arXiv preprint arXiv:2310.11511 (2023)
- [4] Kalai, A.T., Nachum, O., Vempala, S.S., Zhang, E.: Why language models hallucinate. arXiv preprint arXiv:2509.04664 (2025)
- [5] Banerjee, S., Agarwal, A., Singla, S.: Llms will always hallucinate, and we need to live with this. arXiv preprint arXiv:2409.05746 (2024)
- [6] Muennighoff, N., Yang, Z., Shi, W., Li, X.L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E., Hashimoto, T.: s1: Simple test-time scaling. In: arXiv Preprint arXiv:2501.19393 (2025)
- [7] Xie, Y., Kawaguchi, K., Zhao, Y., Zhao, X., Kan, M.-Y., He, J., Xie, Q.: Self-evaluation guided beam search for reasoning. In: Advances in Neural Information Processing Systems (NeurIPS) (2023). <https://arxiv.org/abs/2305.00633>
- [8] Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Durr, P., Murati, L., Radford, A., *et al.*: Language models are few-shot learners. Advances in neural information processing systems **33**, 1877–1901 (2020)
- [9] Wei, J., Tay, Y., Bommasani, R., Bosselut, A., Brown, T., Chen, P., Dang, C., Fan, H.W., Fomicheva, L., Goswami, S., *et al.*: Emergent abilities of large language models. Transactions on Machine Learning Research (TMLR) (2022)
- [10] Venkit, P.N., Chakravorti, T., Gupta, V., Biggs, H., Srinath, M., Goswami, K., Rajtmajer, S., Wilson, S.: An audit on the perspectives and challenges of hallucinations in nlp. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 6528–6548 (2024). <https://aclanthology.org/2024.emnlp-main.375.pdf>
- [11] Islam, S.B., Rahman, M.A., Hossain, K.S.M.T., Hoque, E., Joty, S., Parvez, M.R.: Open-rag: Enhanced retrieval-augmented reasoning with open-source large language models. arXiv preprint arXiv:2403.11264 (2024)
- [12] Talmor, A., Berant, J.: The web as a knowledge-base for answering complex questions. In: Proceedings of the 2018 Conference of the North American Chapter of the ACL (NAACL-HLT), pp. 641–651 (2018). <https://aclanthology.org/N18-1059.pdf>
- [13] Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W.W., Salakhutdinov, R., Manning, C.D.: Hotpotqa: A dataset for diverse, explainable multi-hop question answering.

- In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 2369–2380 (2018). <https://aclanthology.org/D18-1259.pdf>
- [14] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H.: Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997 (2023)
 - [15] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive nlp tasks. arXiv preprint arXiv:2005.11401 (2020)
 - [16] Deng, X., Feng, Y., Zhang, Y., Xu, J., Ma, J., Li, M., Wang, Q.: Re-ranking over multiple reasoning paths improves factual reliability. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 5468–5480 (2023)
 - [17] Wei, J., Tay, Y., Bommasani, R., Zhou, Q.V., Le, Q.: Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems (NeurIPS) (2022)
 - [18] Zibakhsh, S., Samragh, M., Nishu, K., Hannah, L., Kundu, A., Cho, M.: Moes are stronger than you think: Hyper-parallel inference scaling with roe. In: arXiv Preprint (2025). *CHECK_ID : CurrentID2509.17238isaduplicateofgreedy stochastic search*
 - [19] Rajpurkar, P., Zhang, J., Lopyrev, K., Liang, P.: Squad: 100,000+ questions for machine comprehension of text. In: Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (2016)